

NP

La domanda è, ma 3-SAT è P?

Perché è così importante np completezza, $3\text{-SAT} \in P \Leftrightarrow P = NP$.

Intuizione

Tanti problemi hanno la caratteristica di avere un numero esponenziale di soluzioni, ma controllare se una soluzione è corretta richiede P

Esempio 1

3col $\Rightarrow$ Il numero di 3-colorazioni è esponenziale in n , ma controllare una 3-colorazione è corretta richiede tempo polinomiale

Esempio 2

3-SAT $\Rightarrow$ stesso discorso di **3col**

Esempio 3

PATH, 2-SAT $\Rightarrow$ sono in np e abbiamo visto infatti anche in P

In generale per i problemi in NP le soluzioni candidate sono verificabili con stringhe di lunghezza polinomiale

Verificare una soluzione è possibile in tempo polinomiale

Def (Verificatore)

Una TM V è un verificatore per un linguaggio L , se

V ha input $\langle x, y \rangle$

$$\forall x \mid x \in L \Leftrightarrow \exists y \mid V(\langle x, y \rangle) = \text{Acc}$$

Nota 1

Questo significa che abbiamo 2 casi

Yes Case $x \in L \Rightarrow \exists y \mid V(\langle x, y \rangle) = \text{Acc}$

No Case $x \notin L \Rightarrow \forall y, V(\langle x, y \rangle) = \text{Rej}$

Nota 2

V ha tempo di esecuzione polinomiale se il suo tempo di esecuzione è $O(|x|^k)$, per $k \in \mathbb{N}$

Conseguenza

$|y| = \text{poly}(n)$, perché V deve almeno leggere y

Def (Verifier-NP)

NP è l'insieme dei linguaggi L | L ha un verificatore di tempo polinomiale

Vediamo alcuni esempi

Esempio 1

Teorema 1

$3\text{col} \in \text{NP}$

Dim

Basta considerare il seguente verificatore V

su input $\langle x, y \rangle$, dove $x = G$, cioè un grafo

Interpreta $y = \langle c_1, c_2, \dots, c_n \rangle$, come sequenza di colori, con $n = \#$ di vertici in G e

$c_i \in \{R, Y, B\}$

$\forall (i, j) \in E$, rifiuta $\iff c_i = c_j$

↳ archi del grafo

se non ha mai rifiutato, allora accetta

Inanzitutto V ha tempo polinomiale in $|x|$. Inoltre

Yes Case $G \in 3\text{col} \implies \exists$ una 3-colorazione $y = (c_1, \dots, c_n) \mid \forall (i, j) \in E \implies c_i \neq c_j$.
Allora $V(x, y) = \text{Acc}$

No Case Se $V(x, y) = \text{Acc}$, per qualunque $y = (c_1, \dots, c_n) \implies y$ è una 3-colorazione, ovvero $G \in 3\text{col}$.

Esercizio per casa

$3\text{SAT} \in \text{NP}$

Qual è la relazione tra P e NP ?

Teorema 2

$$P \subseteq NP \subseteq EXP$$

Ricordiamo che $P \neq EXP$ dal Teorema di gerarchie di tempo (THT) Ma allora $P \neq NP$ e $NP \neq EXP$ = Tutte e due

Dim

Sia $L \in P$ / $\exists M$ / $L(M) = L$ e M ha tempo polinomiale Basta considerare il verificatore che.

su input $\langle x, y \rangle$

Ignora y e accetta $\Leftrightarrow M(x) = Acc$

Altre inclusioni

Se $L \in NP \Rightarrow \exists V$ che è un verificatore di tempo polinomiale Questo significa come già detto che $|y| = O(|x|^k)$ per $k \in \mathbb{N}$ e quindi $\exists M$ che decide L in tempo esponenziale provando tutti gli y

C'è un'altra definizione di NP **Non-deterministic Polynomial Time**

Qui consideriamo la macchina di Turing non-deterministica

La complessità di Tempo di una NTM è il massimo tempo di esecuzione su tutti i rami di computazione della NTM

Def(NTIME)

Una $NTIME(f(n))$ è l'insieme di linguaggi L / $\exists$ NTM N per cui $L(N) = L$ e N ha tempo $O(f(n))$

Def(NP)

$$NP = \bigcup_{k \in \mathbb{N}} NTIME(n^k)$$

Def(NEXP)

$$NEXP = \bigcup_{k \in \mathbb{N}} NTIME(2^{n^k})$$

Teorema 3

3-SAT $\in$ NP

Dim.

Considero la NTM N che su input $\langle x \rangle = \langle \emptyset \rangle$ ha una 3-SAT formula

Prove non-deterministicamente $x_1, \dots, x_m$ e controlla $\phi(x_1, \dots, x_m) = 1$

Teorema 4

Verifier-NP = NP

Dim

($\Rightarrow$)

Sia L con verificatore V di tempo polinomiale in $|x|$

Devo definire una NTM $N \mid L(N) = L$ Come sappiamo, $V(\langle x, y \rangle)$, dove $|y| = O(|x|^k)$, ovvero $\leq c \cdot x^k$, per qualche costante

Indovino non-deterministicamente y

Esegue deterministicamente $V(\langle x, y \rangle)$ in ogni ramo, e accetta $\Leftrightarrow V(\langle x, y \rangle) = \text{Acc}$

Chiarimento

$x \in L \Leftrightarrow$ almeno un ramo accetta $\Leftrightarrow \exists y \mid V(\langle x, y \rangle) = \text{Acc} \Leftrightarrow N(x)$ accetta

($\Leftarrow$)

Sia $L \in \text{NP}$, ovvero $\exists$ NTM $N \mid L(N) = L$ e N ha tempo polinomiale Devo verificare che V è un verificatore per L di tempo polinomiale

Idea

Il certificato y sarà l'insieme delle scelte non-deterministiche con una stringa di lunghezza polinomiale, perché N ha tempo polinomiale

sull'input $\langle x, y \rangle$

Simula deterministicamente $N(x)$ usando le scelte non-deterministiche contenute in y

Chiarimento

$x \in L \Leftrightarrow N(x)$ accetta $\Leftrightarrow \exists$ un y che è l'insieme di scelte non-deterministiche, per cui N accetta nel relativo ramo di computazione

Ma questo è vero $\Leftrightarrow \exists y \mid V(\langle x, y \rangle) = \text{Acc}$

NP-completetzza

Abbiamo visto che $\exists$ linguaggi $L \in \text{NP}$, come per esempio **3SAT**, che non sembrano essere in P Come vedremo questi linguaggi sono NP-completi.

Abbiamo visto che $\exists$ linguaggi $L \in \text{NP}$, come per esempio SAT , che non sembrano essere in P . Come vedremo questi linguaggi sono NP-completi.

$$L \in P \Leftrightarrow P = NP$$

Come si fa a vedere questo? Tramite le riduzioni.

Def (NP-completezza)

Siano A, B due linguaggi

Diremo che $A \leq_m^P B$ se $\exists$ poly-Time $R: \Sigma^* \rightarrow \Sigma^* \mid \forall x \in \Sigma^*: x \in A \Leftrightarrow R(x) \in B$

Note.

È identico alla precedente definizione di mapping reduction, ma ora R è poly-Time Computabile

Teorema 5 (Mapping reduction)

Se $A \leq_m^P B$ e $B \in P \Rightarrow A \in P$

Dim

Dato $M_B \mid L(M_B) = B$ e M_B è poly-Time, considero la TM

$$M_A() = M_B(R())$$

dove $R()$ è la mapping reduction per $A \leq_m^P B$. Chiusamente M_A è poly-Time, perché R è poly-Time e M_B è poly-Time, quindi la sua complessità è $\text{poly}(\text{poly}(n)) = \text{poly}(n)$

Inoltre

$$x \in A \Leftrightarrow R(x) \in B \Leftrightarrow M_B(R(x)) = \text{Acc} \Leftrightarrow M_A(x) = \text{Acc}$$

Allora $A \in P$